Minesweeper Image Reconstruction — Project Documentation

> **Status:** Active development. Core pipeline implemented and validated through 6 iterations. Large-scale (200×125+ cells) repair performance is the current bottleneck. Not yet production-ready for 2000×2000px input at high fidelity.

---

1. Project Overview

Purpose

This system converts an arbitrary input image into a valid Minesweeper board such that:

1. The **mine placement** is optimized so that the resulting number field (each cell's count of adjacent mines) visually approximates the input image when revealed.
2. The board is **fully solvable using deterministic logic** — no guessing required.
3. The **revealed state** of a completed game reconstructs the original image.

The target use case includes high-resolution anime line-art at up to 2000×2000px input, mapped onto boards of 200×125 cells or larger.

Core Problem Statement

This is a **constrained inverse convolution problem**. The number field N is a convolution of the binary mine grid G with a 3×3 neighborhood kernel. The goal is to find G such that N ≈ T (the target image), subject to Minesweeper solvability constraints.

```
G ∈ {0,1}^(W×H)          — binary mine grid
N(x,y) = Σ G(i,j)        — sum over 8-neighborhood (the number field)
T(x,y) ∈ [0,8]           — target derived from input image
L = Σ w(x,y)(N(x,y) - T(x,y))²   — weighted reconstruction loss
```

Hard Constraints

Constraint	Rule
Number validity	N(x,y) ∈ {0,1,…,8} everywhere
Solvability	Board must be 100% logically deducible (no guessing)
Playability	Every safe cell must be reachable via deterministic inference
No random placement	Mine placement must be optimization-driven

---

2. System Architecture

Component Overview

```
Input Image (PNG/JPG)
        |
        ▼
┌──────────────────────┐
|  Image Preprocessor  |  → Grayscale, panel crop, contrast enhance,
|  load_image_smart()  |    LANCZOS resize, histogram stretch → T ∈ [0,8]
└──────────┬───────────┘
           |
           ▼
┌──────────────────────┐
|  Weight Map          |  → Sobel edge detection → perceptual weight
|  compute_edge_       |    matrix W (edges weighted 4–5×)
|  weights()           |
└──────────┬───────────┘
           |
           ▼
┌──────────────────────┐
|  Smart Initializer   |  → Probabilistic mine seeding from T/8
```

```
│  init_grid()        │    + mine-free border (width=3)
│                     │    + mine-free corridor grid every N rows/cols
└─────────┬───────────┘
          │
          ▼
┌──────────────────────────────────────────────────────┐
│  Multi-Scale Simulated Annealing  multiscale_sa_v2()  │
│                                                       │
│  Phase 1 (Coarse): 50% scale board, 300-800k iters    │
│    → Numba JIT inner loop (_sa_inner)                 │
│    → Incremental N-field updates (O(1) per flip)      │
│    → Hard N∈[0,8] enforcement per move                │
│                                                       │
│  Phase 2 (Fine): Full board, warm-started from upsample │
│    → 1-3M iters, lower T_start (25% of coarse T)      │
│    → Adaptive density cap enforcement                 │
└──────────────────────────────────────────────────────┘
          │
          ▼
┌────────────────────┐
│  CSP Solver        │  → BFS flood-fill from zero-N cells
│  solve_board()     │    → Constraint propagation (flag/reveal)
│                    │    → Subset constraint propagation (A⊂B rules)
│                    │    → Returns: coverage, revealed, flagged, unknown
└─────────┬──────────┘
          │  (if coverage < 99.9%)
          ▼
┌────────────────────┐
│  Targeted Repair   │  → Identify unknown safe cells
│  targeted_repair() │    → Score nearby mines by how many unknowns
│                    │      they affect (priority queue)
│                    │    → Remove highest-impact mines greedily
└─────────┬──────────┘
          │
          ▼
┌────────────────────┐
│  Renderer          │  → Full 6-panel report (target, mines, N field,
│  render_full_      │    error map, solve map, metrics)
│  report()          │    → Hi-res board raster render
│  save_board_hires() │
└────────────────────┘
```

Key Data Structures

Name	Type	Description
`grid`	`np.int8[H,W]`	Binary mine grid (0=safe, 1=mine)
`N`	`np.float32[H,W]`	Number field (0–8 count of adjacent mines)
`target`	`np.float32[H,W]`	Image-derived target values in [0,8]
`weights`	`np.float32[H,W]`	Per-cell loss weights (edge-boosted)
`revealed`	`set of (y,x)`	Cells the solver has logically opened
`flagged`	`set of (y,x)`	Cells the solver has flagged as mines
`unknown`	`set of (y,x)`	Safe cells the solver cannot yet resolve

---

3. Key Design Decisions & Rationale

3.1 Simulated Annealing over Genetic/Brute-Force

SA was chosen because the loss landscape is smooth (small mine flips cause bounded local changes to N), incremental delta computation is O(1) via in-place N updates, and it naturally escapes local minima via temperature schedule. Genetic algorithms were considered but rejected due to crossover complexity with hard N≤8 constraints.

3.2 Incremental N-Field Updates

Rather than recomputing the full convolution after each flip (O(W·H)), only the 8 neighbors of the flipped cell need updating. This reduces per-step cost from O(WH) to O(1) and is the key enabler for Numba JIT achieving >1M iterations/second.

3.3 Numba JIT for Inner SA Loop

The SA inner loop (flip selection, delta computation, accept/reject, N update) is compiled with `@njit(cache=True, fastmath=True)`. On a 120×75 board, this achieves ~**5M iterations/second** vs ~50k/iteration in pure Python. The entire loop is passed arrays by value and operates in-place on copies.

```python
# Benchmark result:
# 1M iters on 120×75 board: 0.51s  (pure Python equivalent: ~20s)
```

3.4 Mine-Free Corridors in Initialization

Standard random seeding creates mine islands disconnected from the border, which the CSP solver cannot reach. The initialization forces mine-free horizontal and vertical corridors at regular intervals (every 8–10 cells), guaranteeing flood-fill connectivity from all four borders into the interior. This alone drove coverage from **1.2% → 87.9%** between Iterations 1 and 2.

3.5 Multi-Scale Coarse→Fine Optimization

Large boards (200×125 = 25,000 cells) are too costly to optimize from scratch at full resolution. The two-phase approach:

1. Optimize at 50% scale (e.g., 100×62) — fast convergence to good global structure
2. Upsample via nearest-neighbor, re-apply corridor/border constraints
3. Fine-tune at full resolution with a warm start and lower temperature

This halves total SA cost while improving final quality by avoiding poor global initialization.

3.6 Edge-Boosted Perceptual Weights

High-contrast edge regions carry the most visual information. A Sobel-filtered weight map multiplies the loss contribution of edge cells by 4–5×, directing optimization toward preserving edge fidelity over flat background regions.

3.7 Fundamental Tension: Visual Quality vs. Solvability

A key insight identified during development: **mine density is an adversarial variable** relative to the two objectives:

Higher density	Effect
↑ Visual quality	More variety in N values → better image approximation
↓ Solvability	More ambiguous number patterns → more unsolvable 50/50s

Empirically, a density cap of **0.20–0.22** (20–22% of cells are mines) provides the best balance. Above ~28%, coverage degrades sharply. Below ~15%, the number field becomes too uniform for useful image reconstruction.

3.8 Two-Phase vs. Joint Optimization (Iteration 5 Lesson)

An attempt was made in Iteration 4 to run SA with periodic solvability checks, reverting moves that broke solvability. This failed: it accepted 0 improvements because any helpful move would temporarily reduce coverage, causing reversion. The correct approach (validated in Iteration 5) is a **joint loss**:

```
L_joint = L_visual + λ(t) · L_solvability
```

where `λ` adapts upward when coverage stagnates and downward when coverage is high. This prevents the adversarial revert loop.

---

4. Iteration History

Iteration 1 — Baseline

Setup: 30×20 synthetic smiley-face target. Pure SA, 200k iterations, geometric cooling.

Problem: Mine density 44.8% → no zero-N regions → solver only opened 1.2% of the board. Temperature cooled too fast; loss stagnated at 2,296.

Lesson: Density must be capped; solver needs zero-N entry points.

Iteration 2 — Density Control + Enhanced Solver

Changes: Density capped at 28%, 2-cell mine-free border, 500k iterations with reheat, subset constraint propagation added to solver.

Result: Coverage jumped from **1.2% → 87.9%**. Mine accuracy 52%. Loss increased to 3,797 (expected — fewer mines means fewer high numbers).

Lesson: Border entry points are critical. Subset propagation meaningfully extends solver reach.

Iteration 3 — Post-SA Repair Pass

Changes: After SA, run a greedy repair loop: for each unsolvable safe cell, find adjacent mines, try removing the highest-impact one, keep the move if it improves coverage.

Result: Coverage **87.9% → 99.5%**, mine accuracy **52% → 91.4%**. Loss degraded to 5,050 (repair removes mines, reducing N values toward target mismatch in bright regions).

Lesson: Repair is effective but loss-destructive. The two objectives need to be balanced earlier, not sequentially patched.

Iteration 4 — Solvability-Constrained SA

Changes: Deeper repair (radius 4, 80 rounds). Loss-recovery SA that periodically checks solvability and reverts moves that break it.

Result: Coverage **99.5% → 99.76%**, mine accuracy 94%. But loss-recovery SA accepted **0 solvable improvements** — reverting to last known solvable state prevented any useful convergence.

Root cause: Sequential "optimize then constrain" approach creates adversarial interference. Any move that improves loss temporarily disrupts solvability → gets reverted.

Iteration 5 — Joint Multi-Objective SA

Changes: Complete redesign. Joint loss with adaptive λ. Mine-free corridor grid seeding at initialization. Adaptive λ increases when coverage plateaus.

Result: Best loss so far (4,384, lower than Iters 3–4), coverage 98.3%, mine accuracy 88.7%.

Remaining gap: ~1.7% of safe cells remain logically indeterminate — classic "50/50" symmetric configurations that are provably unresolvable without guessing at these density levels.

Iteration 6 — Large-Scale Real Image Pipeline

Target image: Anime line-art (550×686px left panel of a figure). Board sizes: 120×75 (9,000 cells) and 200×125 (25,000 cells).

New infrastructure: Numba JIT SA inner loop, multi-scale coarse→fine pipeline, `load_image_smart()` with panel crop and histogram stretch, `save_board_hires()` raster renderer.

Status at cutoff: 120×75 medium scale completed SA (loss 20,808, density 19.8%, coverage 91.9% pre-repair). Repair loop running but hitting timeout — the $O(C^2)$ subset constraint check in `solve_board()` becomes the bottleneck when called repeatedly inside `targeted_repair()`.

Unresolved: 200×125 large scale has not yet completed. Repair performance needs optimization.

---

5. Current Implementation State

Implemented and Working

Module	File	Status
Numba JIT SA core	`sa_core.py`	✅ Complete, benchmarked
Image loader	`large_scale_engine.py` → `load_image_smart()`	✅ Complete
Edge weight map	`compute_edge_weights()`	✅ Complete
Grid initialization	`init_grid()`	✅ Complete
Multi-scale SA	`multiscale_sa_v2()`	✅ Complete
N-field convolution	`compute_N()`	✅ Complete
CSP solver (basic + subset)	`solve_board()`	✅ Complete
Targeted repair	`targeted_repair()`	⚠️ Correct but slow at scale
Full report renderer	`render_full_report()`	✅ Complete
Hi-res board raster	`save_board_hires()`	✅ Complete

Not Yet Complete

- **Large-scale repair performance:** `targeted_repair()` calls `solve_board()` for each candidate mine — at 200×125, the $O(C^2)$ subset propagation in the solver makes each repair candidate check expensive. This must be replaced with an incremental solver.
- **250×250 board scale (62,500 cells):** Not yet tested. Requires the repair fix first.
- **2000×2000px input → 250×250 board:** The image loading pipeline supports it, but end-to-end has not been validated at this scale.
- **Visual reconstruction quality at large scale:** At 120×75, the SA loss (20,808 unweighted) is high. More iterations or a better initialization strategy are needed.

Key Metrics at Current Best State

Scale	Cells	Loss	Coverage	Solvable	Mine Density	Time
30×20 (synth)	600	4,384	98.3%	No	0.325	~15s
120×75 (anime)	9,000	20,808	91.9%*	No	0.198	~5s SA
200×125 (anime)	25,000	TBD	TBD	TBD	TBD	TBD

*Pre-repair. Repair was still running at cutoff.

---

6. Open Questions & Risks

OQ-1: Irreducible 50/50 Ambiguity

At mine densities above ~15%, some mine configurations create locally symmetric number patterns that no deterministic rule can resolve. These are mathematically provably unresolvable. The question is: **how much density must be sacrificed to eliminate them entirely?**

Hypothesis: Dropping density to ~12–15% and using a modified initialization that avoids 2×2 mine clusters may eliminate most 50/50s, but at significant cost to visual quality (most N values will be 0 or 1).

OQ-2: Repair Scalability at Large Boards

Current `targeted_repair()` calls the full CSP solver per candidate mine removal. At 200×125 with 600+ unknown cells and 2,000+ candidate mines, each repair round requires hundreds of full solver invocations. This is O(repairs × candidates × C^2).

Risk: At 250×250 boards, this could take hours. An incremental solver that only re-evaluates affected constraints after a single mine removal is required.

OQ-3: Loss vs. Coverage Tradeoff at Scale

On small boards, loss ~4,000 corresponds to visually recognizable output. At 120×75, loss 20,808 normalized per cell is similar (~2.3 per cell vs. ~7.3 per cell on small boards), but the visual quality hasn't been confirmed. The relationship between normalized loss and perceptual quality needs empirical verification at each scale.

OQ-4: Numba Compilation Overhead

The JIT compilation warmup takes ~5s on first call (then cached). In batch/repeated use this is negligible, but it complicates interactive workflows. The `cache=True` flag mitigates this after the first run.

OQ-5: Corridor Spacing at High Resolution

At 200×125 with `corridor_step=10`, corridors remove ~20% of potential mine cells before optimization starts. This biases the mine distribution toward a regular grid pattern that may create visible horizontal/vertical artifacts in the final number field. Optimal corridor spacing as a function of board size has not been determined.

---

7. Next Steps

Immediate (Unblock Large Scale)

1. Incremental Solver for Repair

Replace the full `solve_board()` call inside `targeted_repair()` with an incremental solver that maintains constraint state between calls and only re-evaluates constraints in the neighborhood of the modified cell.

```python
class IncrementalSolver:
    def __init__(self, grid): ...
    def remove_mine(self, y, x) -> float:  # returns new coverage, O(local)
    def get_coverage(self) -> float
```

2. Vectorized Constraint Check

The O(C^2) subset propagation should be replaced with a sparse constraint index — mapping each cell to the set of constraints it participates in. Updates can then be O(degree) per cell rather than O(C^2) globally.

3. Repair Batching

Instead of removing one mine per round, identify the top-K candidates simultaneously (those touching the most unknown cells), remove a batch, and re-solve once. Accept the batch if coverage improves.

Short-Term (Quality Improvement)

4. Adaptive Corridor Spacing

Scale corridor spacing with board size: `corridor_step = max(6, W // 20)`. Also consider diagonal corridors or a Poisson-disk-style exclusion zone rather than a rigid grid.

5. Perceptual Loss Function

The current L2 loss treats all N values equally. A perceptual weighting that penalizes mismatches at edges more heavily (already partially done via `compute_edge_weights`)

should be extended to include a structural similarity (SSIM) component between the normalized N field and target.

6. Post-Repair Fine-Tuning SA

After repair achieves full solvability, run a short constrained SA pass (similar to Iter 4 but using the joint-loss formulation from Iter 5) to recover loss without breaking solvability. The key difference from Iter 4: only check solvability every 5,000 steps rather than every step, and use a smaller revert window.

Medium-Term (Scale Validation)

7. End-to-End 250×250 Test

Once the incremental solver is implemented, run the full pipeline on a 250×250 board from a 2000×2000px input image. Target metrics:

- Coverage ⩾ 99.5%
- Mean |N−T| ⩽ 1.5
- Total pipeline time ⩽ 10 minutes

8. Quality Evaluation Protocol

Define a reproducible evaluation: run 5 boards from the same image at each scale, report mean and std of loss, coverage, and a human-rated visual similarity score (1–5 scale). This will reveal whether the pipeline is consistent or highly seed-dependent.

Appendix: File Map

```
/home/claude/minesweeper/
├── sa_core.py              # Numba JIT SA inner loop (primary computation)
├── large_scale_engine.py   # All other components (loading, solver, render)
├── core.py                 # Original small-scale implementation (Iters 1-5)
├── iter1.py ... iter6.py   # Per-iteration runner scripts
└── results/
    ├── iter6_medium_report.png    # 6-panel analysis for 120×75
    ├── iter6_medium_board.png     # Hi-res board render for 120×75
    ├── grid_medium.npy            # Best mine grid for 120×75
    ├── target_medium.npy          # Preprocessed target for 120×75
    ├── state_medium.pkl           # Full pipeline state checkpoint
    └── metrics_medium.json        # Metrics for 120×75
```

Appendix: Configuration Reference

```python
# Recommended config for a 200×125 board
config = dict(
    density        = 0.20,          # mine fraction cap
    T_start        = 8.0,           # SA initial temperature
    T_min          = 0.001,         # SA minimum temperature
    alpha_coarse   = 0.99998,       # coarse-pass cooling rate
    alpha_fine     = 0.999995,      # fine-pass cooling rate
    iters_coarse   = 800_000,       # coarse-pass SA iterations
    iters_fine     = 3_000_000,     # fine-pass SA iterations
    coarse_scale   = 0.5,           # fraction of full board for coarse pass
    corridor_step  = 10,            # mine-free corridor spacing (cells)
    border         = 3,             # mine-free border width (cells)
    repair_rounds  = 70,            # max targeted-repair rounds
    seed           = 31,            # RNG seed for reproducibility
)
```

Appendix: Loss Progression Across Iterations (Small Board, Synthetic Target)

Iteration	Key Change	Loss	Coverage	Mine Density
1	Baseline SA	2,296	1.2%	0.448

Iteration	Key Change	Loss	Coverage	Mine Density
2	Density cap + border	3,797	87.9%	0.363
3	Post-SA repair	5,050	99.5%	0.308
4	Deep repair + constrained SA	5,633	99.8%	0.307
5	Joint multi-objective SA	**4,384**	98.3%	0.325

Note: Loss increased from Iter 1–4 because density was reduced (correct trade-off). Iter 5 partially recovered loss while maintaining high coverage by using a joint objective from the start.